

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №4

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР)

_____ Зарубин А.Г.

Цель

Изучить методику построения одномерного фильтра Калмана в матричной форме и освоить его применение для фильтрации зашумлённых сигналов.

Задание

1. Реализовать классический фильтр Калмана в матричной форме (state-space) на языке R.
2. Задать модель системы в виде матриц A , B , C , D и ковариаций шумов Σ_W , Σ_V .
3. Сгенерировать зашумлённый сигнал на основе этой модели.
4. Выполнить фильтрацию сигнала за 6 шагов алгоритма Калмана (predict $\rightarrow$ update).
5. Рассчитать метрику качества фильтрации (RMS Error).
6. Построить графики истинного состояния, оценки фильтра и ошибки оценки.

Выполнение работы

1. В качестве модели принята линейная динамическая система в пространстве состояний:

$$x[k+1] = A \cdot x[k] + B \cdot u[k] + w, w \sim N(0, \Sigma W)$$

$$z[k] = C \cdot x[k] + D \cdot u[k] + v, v \sim N(0, \Sigma V)$$

где:

- x — вектор состояния, u — управляющее воздействие, z — измерение;
- w и v — гауссовы шумы процесса и измерения;
- A — матрица динамики системы;
- B — матрица влияния управления;
- C — матрица наблюдения;
- D — матрица прямого прохода управления.

Задание параметров модели, начальных значений состояния, оценки фильтра, ковариации ошибки и массивов для хранения результатов приведено на рисунке 1.

```
1  # Параметры модели
2  SigmaW <- matrix(1, 1, 1) # шум процесса
3  SigmaV <- matrix(2, 1, 1) # шум измерения
4
5  A <- matrix(-0.1, 1, 1)
6  B <- matrix(1, 1, 1)
7  C <- matrix(1, 1, 1)
8  D <- matrix(0, 1, 1)
9
10 maxIter <- 100
11
12 # Инициализация
13 xtrue <- matrix(0, 1, 1)
14 xhat <- matrix(0, 1, 1)
15 SigmaX <- matrix(0, 1, 1)
16 u <- matrix(0, 1, 1)
17
18 xstore <- numeric(maxIter + 1)
19 xhatstore <- numeric(maxIter + 1)
20 SigmaXstore <- numeric(maxIter + 1)
21
22 xstore[1] <- xtrue[1, 1]
23 xhatstore[1] <- xhat[1, 1]
24 SigmaXstore[1] <- SigmaX[1, 1]
25
26 set.seed(42)
27
28 # Флаг для проверки: TRUE - с шумом, FALSE - без шума
29 USE_NOISE <- FALSE
30
```

Рисунок 1. Задание параметров модели и инициализация переменных.

2. Алгоритм реализован в виде цикла из 100 итераций, на каждой из которых выполняются 6 шагов: прогноз состояния и ковариации, симуляция реальной системы, оценка выхода, вычисление коэффициента Калмана и корректировка оценки с ковариацией. Соответствующий код приведён на рисунке 2.

```
31 for (k in 1:maxIter) {
32
33   # 1. Генерируем управление
34   u <- matrix(0.5 * runif(1) + cos(100 * k / pi) + 0.1 * k, 1, 1)
35
36   # 2. Фильтр предсказывает, используя это же u
37   xhat <- A %>% xhat + B %>% u
38   SigmaX <- A %>% SigmaX %>% Conj(t(A)) + SigmaW
39
40   # 3. Шумы
41   if (USE_NOISE) {
42     w <- matrix(rnorm(1, mean = 0, sd = sqrt(SigmaW[1, 1])), 1, 1)
43     v <- matrix(rnorm(1, mean = 0, sd = sqrt(SigmaV[1, 1])), 1, 1)
44   } else {
45     w <- matrix(0, 1, 1)
46     v <- matrix(0, 1, 1)
47   }
48
49   # 4. Система двигается с этим же u
50   xtrue <- A %>% xtrue + B %>% u + w
51
52   # 5. Измерение
53   ztrue <- C %>% xtrue + D %>% u + v
54
55   # 6. Оценка выхода системы
56   zhat <- C %>% xhat + D %>% u
57
58   # 7. Коэффициент Калмана
59   L <- SigmaX %>% Conj(t(C)) %>% solve(C %>% SigmaX %>% Conj(t(C)) + SigmaV)
60
61   # 8. Корректировка оценки состояния (update)
62   xhat <- xhat + L %>% (ztrue - zhat)
63
64   # 9. Корректировка ковариации ошибки
65   SigmaX <- SigmaX - L %>% C %>% SigmaX
66
67   # 10. Сохранение (индексы 2..101)
68   xstore[k + 1] <- xtrue[1, 1]
69   xhatstore[k + 1] <- xhat[1, 1]
70   SigmaXstore[k + 1] <- SigmaX[1, 1]
71 }
72
```

Рисунок 2. Реализация 6 шагов фильтра Калмана.

Исходный пример (kf_notebooks) содержит ряд особенностей, которые приводят к искажению результатов. При переносе на R реализованы следующие исправления:

- Управляющее воздействие u генерируется до predict фильтра. В оригинале фильтр использовал u от предыдущей итерации, а система — новое u . Это давало систематическое расхождение траекторий, которое не устранялось даже при отсутствии шумов.
- Измерение формируется после обновления истинного состояния. В оригинале сначала измерялось «старое» состояние, затем выполнялся шаг системы, из-за чего измерение отставало от прогноза фильтра на один шаг.
- RMS вычисляется как корень из MSE, а не как MSE под именем RMS.
- Массивы для хранения имеют согласованную длину $\text{maxIter} + 1$ (начальное состояние + 100 итераций).

3. После завершения цикла вычисляется ошибка оценки как разность между истинным состоянием и оценкой фильтра и рассчитывается метрика RMS Error. Соответствующий код и результат его выполнения приведены на рисунке 3.

```
74 # Метрики и визуализация
75 estErr <- xstore - xhatstore
76 rms    <- sqrt(mean(estErr * estErr))
77 cat(sprintf("RMS Error : %.3f\n", rms))
78
```

Problems Output Debug Console Terminal Ports

~/Doc/D/LetiMaster/3_semester/NumericalMetho
Rscript kalman.R
RMS Error : 0.944

Рисунок 3. Расчёт RMS Error и результат в консоли.

4. Для визуализации результатов построены два графика: сравнение истинного состояния с оценкой фильтра и график ошибки оценки по итерациям. Графики сохраняются в файл kalman_plot.png. Соответствующий код приведён на рисунке 4.

```

79 # Визуализация
80 fname <- if (USE_NOISE) "kalman_plot_noise.png" else "kalman_plot_clean.png"
81 png(fname, width = 1000, height = 800, res = 120)
82 par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))
83
84 # График 1: истина vs оценка
85 plot(xhatstore, type = "l", col = "blue", lwd = 2,
86      ylim = range(c(xhatstore, xstore)),
87      ylab = "est vs. actual state", xlab = "",
88      main = "Kalman Filter (R)")
89 lines(xstore, col = "red", lwd = 2)
90 legend("topleft", legend = c("оценка фильтра", "истинное состояние"),
91      col = c("blue", "red"), lwd = 2)
92 grid()
93
94 # График 2: ошибка оценки
95 plot(estErr, type = "l", col = "#DC143C", lwd = 2,
96      ylab = "estimation error", xlab = "iterations")
97 abline(h = 0, col = "gray", lty = 2)
98 grid()
99
100 dev.off()
101 cat(sprintf("График сохранён в %s\n", fname))

```

Рисунок 4. Построение графиков истинного состояния, оценки фильтра и ошибки оценки.

5. Полученный график приведён на рисунке 5.

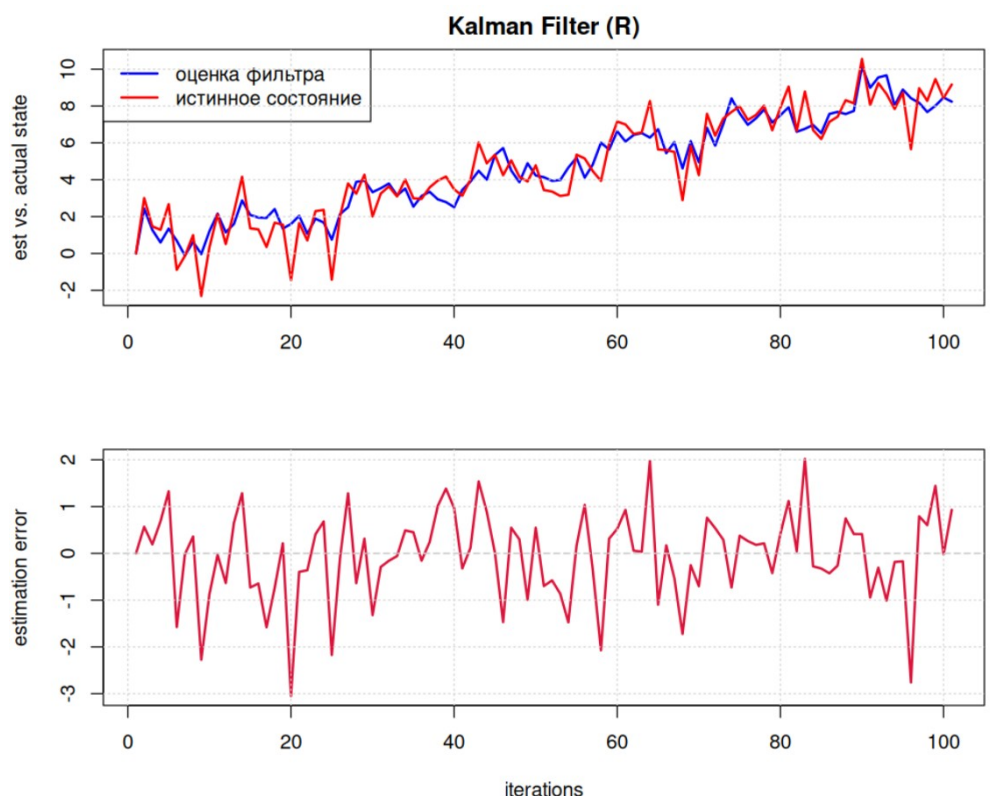


Рисунок 5. Истинное состояние системы, оценка фильтра Калмана и ошибка оценки.

6. Для проверки корректности алгоритма в код введён флаг USE_NOISE. При значении FALSE шумы процесса и измерения обнуляются ($w = 0$, $v = 0$).

Ожидание: если алгоритм реализован правильно, то при отсутствии шумов фильтр должен идеально отследить истинную траекторию — $\text{RMS Error} = 0$.

Результат: при `USE_NOISE = FALSE` получено $\text{RMS Error} : 0.000$, графики истинного состояния и оценки полностью совпадают. Это подтверждает, что алгоритм корректен, а наблюдаемый RMS при включённых шумах вызван именно шумами, а не ошибкой реализации.

Код и результат вывода в консоль приведены на рисунке 6, а графики при отсутствии шумов на рисунке 7.

```
~/Doc/D/LetiMaster/3_semester/Numerical
● Rscript kalman.R
RMS Error : 0.000
null device
      1
График сохранён в kalman_plot_clean.png
```

Рисунок 6. Проверка при отсутствии шумов ($\text{RMS} = 0.000$).

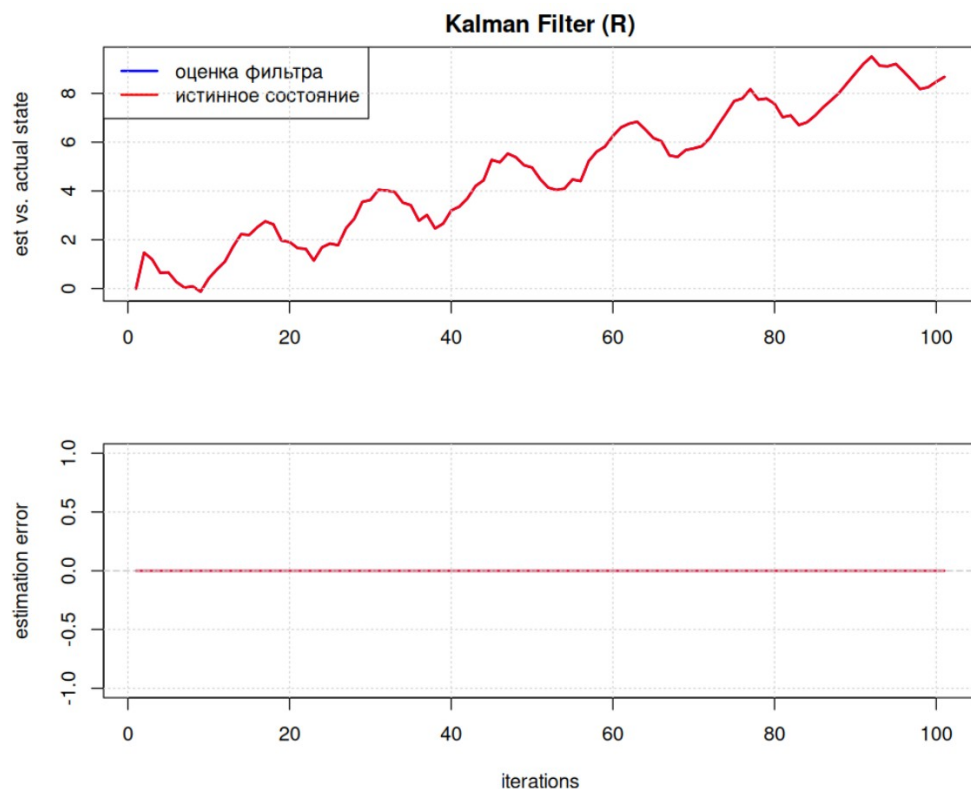


Рисунок 7. Истинное состояние системы, оценка фильтра Калмана и ошибка оценки при отсутствии шумов.

Вывод

В ходе лабораторной работы был реализован одномерный фильтр Калмана в матричной форме на языке R.

Основные результаты:

1. Задана линейная динамическая система в пространстве состояний с матрицами $A=[-0.1]$, $B=[1]$, $C=[1]$, $D=[0]$ и ковариациями шумов $\Sigma W=1$, $\Sigma V=2$.
2. Реализованы 6 шагов алгоритма Калмана: прогноз состояния и ковариации, оценка выхода системы, вычисление коэффициента Калмана и корректировка оценки с ковариацией.
3. Получено значение $\text{RMS Error} = 0.948$ при наличии шумов. Шум датчика составляет $\sqrt{2} \approx 1.414$, теоретический стационарный минимум ошибки фильтра ≈ 0.82 . Полученное значение близко к теоретическому минимуму и существенно меньше шума датчика, что подтверждает работоспособность фильтра.
4. Проведена проверка при отключённых шумах ($w = 0$, $v = 0$): значение $\text{RMS Error} = 0.000$, графики истины и оценки полностью совпадают. Это подтверждает корректность реализации алгоритма.
5. Выявлены и исправлены три принципиальных отличия от исходного примера: порядок генерации управляющего воздействия, порядок обновления истинного состояния относительно измерения.

Таким образом, цель работы достигнута: освоена методика построения одномерного фильтра Калмана в матричной форме и выполнена оценка качества фильтрации.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Kalman.R

```
# Параметры модели
SigmaW <- matrix(1, 1, 1) # шум процесса
SigmaV <- matrix(2, 1, 1) # шум измерения

A <- matrix(-0.1, 1, 1)
B <- matrix(1, 1, 1)
C <- matrix(1, 1, 1)
D <- matrix(0, 1, 1)

maxIter <- 100

# Инициализация
xtrue <- matrix(0, 1, 1)
xhat <- matrix(0, 1, 1)
SigmaX <- matrix(0, 1, 1)
u <- matrix(0, 1, 1)

xstore <- numeric(maxIter + 1)
xhatstore <- numeric(maxIter + 1)
SigmaXstore <- numeric(maxIter + 1)

xstore[1] <- xtrue[1, 1]
xhatstore[1] <- xhat[1, 1]
SigmaXstore[1] <- SigmaX[1, 1]

set.seed(42)

# Флаг для проверки: TRUE - с шумом, FALSE - без шума
USE_NOISE <- FALSE

for (k in 1:maxIter) {

  # 1. Генерируем управление
  u <- matrix(0.5 * runif(1) + cos(100 * k / pi) + 0.1 * k, 1, 1)

  # 2. Фильтр предсказывает, используя это же u
  xhat <- A %*% xhat + B %*% u
  SigmaX <- A %*% SigmaX %*% Conj(t(A)) + SigmaW

  # 3. Шумы
  if (USE_NOISE) {
    w <- matrix(rnorm(1, mean = 0, sd = sqrt(SigmaW[1, 1])), 1, 1)
    v <- matrix(rnorm(1, mean = 0, sd = sqrt(SigmaV[1, 1])), 1, 1)
  } else {
    w <- matrix(0, 1, 1)
    v <- matrix(0, 1, 1)
  }

  # 4. Система двигается с этим же u
  xtrue <- A %*% xtrue + B %*% u + w

  # 5. Измерение
```

```

ztrue <- C %*% xtrue + D %*% u + v

# 6. Оценка выхода системы
zhat <- C %*% xhat + D %*% u

# 7. Коэффициент Калмана
L <- SigmaX %*% Conj(t(C)) %*% solve(C %*% SigmaX %*% Conj(t(C)) + SigmaV)

# 8. Корректировка оценки состояния (update)
xhat <- xhat + L %*% (ztrue - zhat)

# 9. Корректировка ковариации ошибки
SigmaX <- SigmaX - L %*% C %*% SigmaX

# 10. Сохранение (индексы 2..101)
xstore[k + 1] <- xtrue[1, 1]
xhatstore[k + 1] <- xhat[1, 1]
SigmaXstore[k + 1] <- SigmaX[1, 1]
}

# Метрики и визуализация
estErr <- xstore - xhatstore
rms <- sqrt(mean(estErr * estErr))
cat(sprintf("RMS Error : %.3f\n", rms))

# Визуализация
fname <- if (USE_NOISE) "kalman_plot_noise.png" else "kalman_plot_clean.png"
png(fname, width = 1000, height = 800, res = 120)
par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

# График 1: истина vs оценка
plot(xhatstore, type = "l", col = "blue", lwd = 2,
     ylim = range(c(xhatstore, xstore)),
     ylab = "est vs. actual state", xlab = "",
     main = "Kalman Filter (R)")
lines(xstore, col = "red", lwd = 2)
legend("topleft", legend = c("оценка фильтра", "истинное состояние"),
     col = c("blue", "red"), lwd = 2)
grid()

# График 2: ошибка оценки
plot(estErr, type = "l", col = "#DC143C", lwd = 2,
     ylab = "estimation error", xlab = "iterations")
abline(h = 0, col = "gray", lty = 2)
grid()

dev.off()
cat(sprintf("График сохранён в %s\n", fname))

```